

**FINAL PROJECT** Dept. of CSE, BUET

# 2D Positional-Audio Sandbox

---

Team ABC – CSE 220

Sabir Siam : 2305061

Anindya Nag Arghya : 2305086

# Project at a Glance

---

A real-time **2D positional-audio engine**.

Place a listener and sources on a grid, paint walls, and hear – live – how distance, obstruction, material, and reflection change the sound:

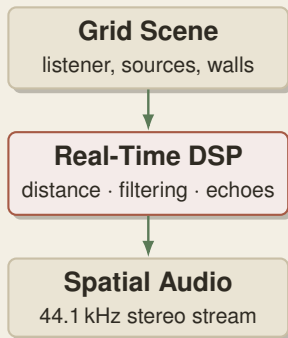
- **Muffling** as paths get obstructed
- **Panning** as sources move left / right
- Delayed, directional **echoes**
- Sound **bleeding through** walls

PYTHON

PYGAME

NUMPY

SOUNDDEVICE



*Live, interactive · all in Python*

# Motivation: Beyond Empty Space

## Free Space is Trivial

- Moving away  $\rightarrow$  volume drops (gain  $\propto 1/d$ )
- Moving sideways  $\rightarrow$  stereo pan (left / right)

In a void, spatial audio is just basic amplitude scaling.

## Real Rooms Have Geometry

- **Corners & edges** – diffract and muffle highs
- **Solid walls** – absorb, let muffled sound bleed through
- **Surfaces** – generate delayed, directional echoes

### The Central Challenge

*“Can we simulate walls, materials, and echoes convincingly, live, using **only** the signal-processing tools this course taught us?”*

**IN SCOPE** Fourier, DFT, FFT, convolution

**OUT OF SCOPE** Laplace/Z-transforms, IIR cascades

Goal: a real-time (44.1 kHz) interactive sandbox, built from FFT multiplication, delay lines, and 2D grid search.

# Audio Input Pipeline: Three Operational Modes

---

## MODE 1: AUDIO FILE

**Format:** .wav, .flac, .mp3

**Pipeline:** Eagerly loaded into float32 NumPy memory

**Role:** Instant testing with no audio hardware needed

## MODE 2: LOCAL MIC

**Format:** Live microphone hardware

**Pipeline:** sounddevice circular stream buffer

**Role:** Real-time voice testing directly in the room

## MODE 3: NETWORK UDP

**Format:** Remote Wi-Fi/Ethernet clients

**Pipeline:** UDP port 50007 to thread-safe queues

**Role:** Multi-source relay with client auto-reclaim

## HARDWARE VERIFIED

**Live Multi-Device Validation:** Mode 3 verified across physical laptops on an active LAN, firewall traversal included.

# Networked Streaming & Sampling Parameters

## Network Plumbing (`network_input.py`)

- **Transport:** UDP port 50007 – drops packets over buffer stalls
- **Packet format:** 32-bit float PCM, 1024 samples per datagram
- **Lifecycle:** background listener → queues; idle timeout  $\approx 5$  s

## Sampling & Latency Budget

- **Nyquist–Shannon:**  $f_{\max} \approx 20$  kHz  
 $\Rightarrow f_s = 44.1$  kHz (no aliasing)
- **Block size**  $N = 1024$ :  
 $\Delta t = N/f_s \approx 23.2$  ms per buffer
- Short enough to feel instant, long enough for stable FFTs



# Challenge 1: The Intuitive Starting Point

---

*“Let’s start with the absolute basics: if a sound source is farther away, it should sound quieter. If it moves to your left, you should hear it in your left ear.”*

## THE CHALLENGE

How do we translate simple 2D grid coordinates  $(x, y)$  into natural volume attenuation and stereo balance — without complex 3D physics?

- **Distance:** amplitude must decay smoothly – no sudden dropoffs, no infinite volume at zero range
- **Direction:** horizontal displacement must pan smoothly without losing volume when centered

# Challenge 1: Distance & Direction — Formulas

## INVERSE-DISTANCE Distance attenuation

$$g(d) = \frac{1.0}{1.0 + k \cdot d}$$

$k = 0.08$  – tuned by ear on a  $40 \times 24$  grid.

## STEREO PAN Linear azimuthal panning

$$\text{pan} = \text{clip}\left(\frac{\Delta x}{20.0}, -1, 1\right), \quad \Delta x = x_{\text{src}} - x_{\text{lis}}$$

$$g_L = \frac{1 - \text{pan}}{2}, \quad g_R = \frac{1 + \text{pan}}{2}$$

## COMBINED STEREO OUTPUT Per-sample signal vector

$$\mathbf{y}[n] = \begin{bmatrix} g_L g(d) x[n] \\ g_R g(d) x[n] \end{bmatrix}$$

### Key Properties

- Constant total gain:  $g_L + g_R = 1$  – no volume dip at center
- Smooth decay: no singularity at  $d=0$
- Pure NumPy: `mono * gain * pan`

# Challenge 1: Signal Scaling in Practice

## SYLLABUS TIE-IN: SCALAR MULTIPLICATION

### The fundamental operation:

$$y[n] = A \cdot x[n]$$

- Time-domain scalar multiplication – the simplest linear transform
- Spatial audio starts as amplitude modulation across two channels
- This is the *baseline* before any frequency-domain or delay-based effects

## IMPLEMENTATION

### Implementation Details

Pure NumPy vectorization: `mono * gain * pan`.  
No per-sample Python loops – entire 1024-sample block processed in one NumPy broadcast.

### Live Demo Cue

**DEMO CUT** Move a single source circularly around the stationary listener on an open grid – volume and stereo balance shift live.

## Challenge 2: Obstacles & Acoustic Character

---

*“A wall shouldn’t just be an ON/OFF mute switch. In real life, when someone speaks behind a door, you still hear them — it just sounds duller, muffled, like the treble was stripped away.”*

### THE CHALLENGE

How do we simulate acoustic “muffling” dynamically in real time — changing the spectral character of the signal, rather than just cutting its volume?

- Sound doesn’t just stop; high frequencies are absorbed far faster than low frequencies
- We need a lowpass filter whose cutoff adapts dynamically to obstacles

# Filtering: The Syllabus Dilemma (IIR vs. FFT)

## REJECTED: BUTTERWORTH IIR

- Standard approach: 2nd-order Butterworth lowpass via `scipy.signal.sosfilt`
- **Problem:** Butterworth design requires the **Z-transform** (bilinear transform, pole stability)
- Z-transforms are taught **weeks later** – defending IIR before covering the theory was a non-starter

## ADOPTED: PER-BLOCK FFT FILTER

- What the syllabus *did* teach: DFT, FFT, frequency-domain multiplication
- **Decision:** build a custom frequency-domain filter from scratch with `rfft/irfft`

### The Academic Philosophy

*Better to implement a first-principles FFT filter we can rigorously defend than drop in an unexplained IIR library.*

# Per-Block FFT Lowpass: The Raised-Cosine Filter

**PER-BLOCK FFT** **RAISED-COSINE** Transform to frequency domain:

$$X[k] = \text{rfft}(x[n]), \quad k = 0, \dots, N/2$$

Raised-cosine gain curve  $H(f)$ , 500 Hz transition band:

$$H(f) = \begin{cases} 1.0 & f \leq f_{\text{low}} \\ 0.5 \left( 1 + \cos \left( \pi \frac{f - f_{\text{low}}}{\Delta f} \right) \right) & f_{\text{low}} < f < f_{\text{high}} \\ 0.0 & f \geq f_{\text{high}} \end{cases}$$

Reconstruct:

$$y[n] = \text{irfft}(X[k] \cdot H[k])$$

**FREQUENCY-DOMAIN MULTIPLICATION**  
**Syllabus tie-in**

- Convolution theorem: multiplication in frequency domain  $\equiv$  convolution in time domain
- Smooth cosine taper avoids Gibbs ringing at the cutoff edge
- Cutoff  $f_c$  is dynamically set by the geometry engine each audio block

**DFT** **CONVOLUTION THEOREM**

# FFT Lowpass: No Overlap-Add — A Deliberate Choice

## WHY NO OVERLAP-ADD? Standard DSP calls

for 50% overlap-add to prevent circular-convolution wraparound. We deliberately omitted it:

- **Short effective impulse response:**

$$L_{\text{eff}} \approx \frac{2}{\Delta f} f_s = \frac{2}{500} \cdot 44100 \approx 176 \text{ samples}$$

- $176 \ll 1024$  (block size) – wraparound artifacts are negligible
- **State simplicity:** avoids overlap-buffer bookkeeping as sources move dynamically

## ENGINEERING SUMMARY

- No Z-transform mathematics required
- 176-sample effective impulse  $\ll$  1024-sample block
- Fully defensible with DFT / FFT theory from the syllabus

## 176-SAMPLE IMPULSE

## NO Z-TRANSFORM

## Live Demo Cue

### DEMO CUT

Toggle a wall between source and listener – immediate treble muffling; watch the spectrum visualizer's high-frequency bins collapse live.

## Challenge 3: Quantifying Obstruction

---

*“A thin barrier directly in front of you shouldn’t sound identical to a sound that has to wind through a maze of three hallways to reach your ears.”*

### THE CHALLENGE

Sound waves diffract and bend around corners. How do we quantify “how obstructed” a path is in arbitrary grid geometry, and map that directly to our FFT filter?

- Sound finds the shortest open path around barriers – not just the Euclidean line
- Every direction change (corner diffraction) strips additional high-frequency energy

# Acoustic Diffraction via 8-Directional Pathfinding

## YEN'S + A\* SEARCH Routing on the 2D

**grid** (`pathfinding.py`)

- **8-directional:** costs 1.0 (ortho) /  $\sqrt{2}$  (diag) – approximates Euclidean acoustics
- **Yen's  $k$ -shortest loopless paths** with A\* (octile heuristic) for the primary acoustic path
- **Corner counting:** recorded whenever step direction  $\Delta \mathbf{p}$  changes

Algorithmic technique applied directly to acoustic modeling.

## CORNER-DRIVEN LOWPASS Mapping

**geometry to DSP** (`dsp_engine.py`)

$$f_c = \max(f_{\min}, f_{\text{base}} - \text{corners} \cdot \Delta f_{\text{corner}})$$

- $f_{\text{base}} = 4000$  Hz (open / 0 corners)
- $\Delta f_{\text{corner}} = 900$  Hz per corner
- $f_{\min} = 300$  Hz (prevents silence)

**Distance gain uses routed path length:**

$$g(d_{\text{path}}) = \frac{1.0}{1.0 + 0.08 \cdot d_{\text{path}}}$$

# Engineering Insight: The “Phantom Corner” Bug

## SUBTLE BUG: PHANTOM CORNERS

- **Symptom:** empty-grid diagonal paths sounded muffled
- **Root cause:** discrete raster steps along non-45° diagonals stair-step between ortho/diag moves
- Pathfinder counted each stair-step as a direction change  $\implies$  3–5 phantom corners in open space
- Caught by ear; confirmed by logging path vectors

## THE FIX: LoS BYPASS

- Bresenham ray-cast:  
`has_line_of_sight(src, lis, walls)`
- **Rule:** if straight line is clear, bypass corners entirely:  $f_c = f_{\text{base}} = 4000 \text{ Hz}$
- Crisp highs in open space; corner muffling only behind real walls

## Live Demo Cue

**DEMO CUT** Compare two sources at equal distance: clear LoS vs. behind a wall corner. Listen for the brightness drop.

## Challenge 4: Sound Takes Time to Travel

---

*“If sound bounces off a distant wall, it shouldn’t just be quieter. It travels a longer physical path, so it has to arrive later — like a real echo.”*

### THE CHALLENGE

Sound has a finite propagation speed. How do we model physical travel time in discrete-time digital audio without bogging down real-time performance?

- Instantaneous attenuation alone sounds unnatural and flat
- Reflected waves must arrive later, proportional to their extra travel distance

⇒ Next: translating physical distance into discrete-time sample shifts using circular buffers.

# Discrete-Time Delay: The Operator

**DISCRETE DELAY**  $x[n - D]$  **Signals tie-in:**  
**the delay operator**

$$y[n] = x[n - D]$$

**Continuous** → **discrete mapping:**

$$\tau = \frac{\Delta d}{c_{\text{sound}}}, \quad D = \text{round}(\Delta d \cdot S_{\text{unit}})$$

**Engine parameters** ( $f_s = 44.1$  kHz):

- $S_{\text{unit}} = 40$  samples/grid-unit ( $\approx 0.907$  ms)
- $D_{\text{max}} = 8000$  samples ( $\approx 181$  ms cap)
- Natural early reflections at room scale

**SYLLABUS TIE-IN** **The time-shift property**

- $x[n - D]$  is one of the fundamental discrete-time operations
- Combined with scaling  $A \cdot x[n - D]$ , it models both attenuation *and* propagation delay
- Foundation for echo modeling in Challenges 6–7

**40 SAMPLES/UNIT**

$f_s = 44.1$  kHz

# Circular Ring Buffer Implementation

## PERSISTENT CIRCULAR BUFFER Audio

arrives in chunks ( $N=1024$ ) – storing full history is impossible.

**Fixed-size circular NumPy array** in `filter_state`, persistent write head  $w_{\text{pos}}$ :

$$\text{write\_idx} = (w_{\text{pos}} + n) \bmod N_{\text{buf}}$$

$$\text{read\_idx} = (w_{\text{pos}} + n - D) \bmod N_{\text{buf}}$$

Zero allocation per block  $\implies$  predictable  $\mathcal{O}(N)$ .

## ENGINEERING PROPERTIES

- Pre-allocated at source creation – no runtime `malloc`
- Modular arithmetic handles wraparound seamlessly
- Same buffer structure reused for all 4 echo slots (Challenge 6)

## Live Demo Cue

**DEMO CUT** Place a source near a reflective wall; trigger a clap — hear the direct sound followed by a distinct, delayed secondary arrival.

## Challenge 5: Total Occlusion & The Snapping Problem

---

*“What happens when you close the last door and completely box a sound in? It shouldn’t suddenly snap to dead silence. Real walls let sound bleed straight through.”*

### THE CHALLENGE

When a room is fully sealed and no open path exists, how do we model acoustic penetration continuously — without jarring volume jumps or artificial floor hacks?

- Sound doesn’t only bend around edges; acoustic energy penetrates directly through solid matter
- Placing the final wall tile must produce a smooth, continuous fade — not an abrupt snap

# Total Occlusion: Flat Floor vs. Physical Transmission

## LEGACY FLAW: THE VOLUME SNAP

- **The shortcut:** 0 open paths  $\rightarrow$  flat constant floor:  
 $\text{gain}_{\text{floor}} \approx 0.02$  ( $-34$  dB)
- **Perceptual failure:** closing the final gap violently snapped from routed audio to  $-34$  dB. Unphysical and jarring.

## FAMILY B: DIRECT TRANSMISSION

- **A distinct arrival:** energy traveling the straight Bresenham line through wall cells
- **Physical complement:**  
 $\tau_{\text{transmission}} = 1.0 - \alpha_{\text{reflectivity}}$
- **Always active:** when sealed, routed audio drops to 0 and transmission takes over smoothly

## The Architectural Fix

*"No open path" no longer snaps to a floor. Only transmission contributes, yielding a perfectly continuous transition.*

# Through-Wall Transmission: Mathematics & Materials

## MULTIPLICATIVE ATTENUATION Bresenham

**line walk** traces all crossed wall cells.

**Compound transmission gain:**

$$g_B = \frac{1.0}{1.0 + 0.08 \cdot d_{\text{Euclid}}} \prod_{w \in \text{crossed}} (1.0 - \alpha_w)$$

- Concrete ( $\alpha=0.90$ )  $\implies$  transmits 10%
- Curtain ( $\alpha=0.25$ )  $\implies$  transmits 75%
- Two curtains:  $(0.75)^2 \approx 56\%$  – 5 $\times$  more than one concrete wall

## STEEPER LOWPASS CUTOFF

Solid barriers absorb highs more aggressively than air diffraction:

$$f_{C,B} = \max(300, 1500 - 500 \cdot N_{\text{walls}}) \text{ Hz}$$

**Zero extra delay:** straight geometric line  
 $\implies$  no delay line needed.

## THROUGH-WALL

## NO SNAP

## PRODUCT $(1-\alpha)$

## Live Demo Cue

### DEMO CUT

Box in a source – sound bleeds through, dark and muffled. Swap concrete for curtains to hear transmission brighten live.

## Challenge 6: One Echo Isn't a Room

---

*“A real room doesn't produce just one echo. If you clap between two walls, you hear sound bouncing off the left wall, the right wall, and the back corner — each arriving at a different time, from a different direction.”*

### THE CHALLENGE

Our early model tracked only a single secondary path ( $k=2$ ). How do we discover, rank, and simulate multiple physically distinct reflections in real time?

- We want geometrically grounded early reflections off visible surfaces — not a generic reverb blur
- Each reflecting wall must sound like an independent source with its own delay, muffle, and angle

# Family C: 12-Ray Radial Candidate Search

## 12-RAY RADIAL FAN The radial ray fan

(pathfinding.py)

- **360° sweep:** 12 rays at 30° increments from the source
- **Outward march:** steps out up to 20-cell radius until hitting a solid wall
- **Explicable:** every echo maps to a visible wall on the grid – no opaque reverb

**DUAL LINE-OF-SIGHT FILTER** A wall qualifies as a true reflector only if **two** rays are clear:

- Source → Wall unobstructed
- Wall → Listener unobstructed

**Predicted loudness:**

$$L(c) = \frac{1.0}{1.0 + 0.08 \cdot 2 d_{\text{wall}}} \cdot \alpha_{\text{material}}$$

**Top-4 selection:** ranked by  $L(c)$ ; best  $M \leq 4$  sent to DSP.

# Multi-Echo Delay Network

## PARALLEL DELAY NETWORK Four

**persistent ring buffers** (`dsp_engine.py`)

- `echo_delay_buf_0...buf_3` in `filter_state`
- **Round-trip delay:**  
 $D_i = \text{round}(2 d_{\text{wall},i} \cdot 40)$  samples
- **Echo gain:**  $g_{C,i} = \frac{\alpha_i}{1.0 + 0.08 \cdot 2 d_{\text{wall},i}}$

## MATERIAL-DEPENDENT CUTOFF Each echo

gets its own lowpass based on the reflecting wall's material:

$$f_{C,i} = \max(300, 3500 - (1 - \alpha_i) \cdot 3000) \text{ Hz}$$

- Hard wall ( $\alpha = 0.90$ ): bright, crisp echo
- Soft curtain ( $\alpha = 0.25$ ): heavily damped echo
- Each echo is independently filtered, delayed, and gained

# Echo Spatial Panning & Demo

**PER-WALL AZIMUTHAL PANNING** Each echo pans toward its reflecting wall, not an averaged direction:

$$\Delta x_i = x_{\text{wall},i} - x_{\text{listener}}$$

**Syllabus tie-in — multi-tap delay filter:**

$$y[n] = \sum_{i=1}^M g_i \cdot x[n - D_i]$$

Each tap corresponds to a physically distinct reflecting surface.

**4 INDEPENDENT ECHOES** Every echo is a fully independent arrival:

- Its own delay (round-trip distance)
- Its own gain (distance + material)
- Its own lowpass (material cutoff)
- Its own stereo pan (wall direction)

## Live Demo Cue

**DEMO CUT** Place a source between a concrete wall (left) and a wood wall (right) — hear two distinct, spatially separated echoes.

## Challenge 7: Bringing It All Together

---

*“We now have a primary diffracted arrival, through-wall transmission bleed, and up to four independent echoes. How do all these separate physical paths combine into a single stereo signal?”*

### THE CHALLENGE

How do we merge disparate acoustic phenomena into one coherent output — and what prevents the combined signal from distorting when multiple loud sources mix?

- Do we need a complicated blending heuristic to glue these arrivals together?
- When multiple sources sum, how do we guarantee signal stability?

# The Culmination: Linear Superposition

**SUMMED ACOUSTIC FAMILIES** The complete  
three-family model (`dsp_engine.py`)

$$\mathbf{y}_{\text{total}}[n] = \mathbf{y}_A[n] + \mathbf{y}_B[n] + \sum_{i=1}^M \mathbf{y}_{C,i}[n]$$

- $\mathbf{y}_A$  – routed path (corner lowpass + distance + LoS bypass)
- $\mathbf{y}_B$  – straight-line transmission (barrier lowpass + material product)
- $\mathbf{y}_{C,i}$  – up to 4 echoes (per-wall delay + muffle + pan)

No global re-normalization — inverse-distance laws already scale each contribution.

**SUPERPOSITION PRINCIPLE** The defining  
law of linear systems:

$$\mathcal{T}\{x_1 + x_2\} = \mathcal{T}\{x_1\} + \mathcal{T}\{x_2\}$$

- Each path is an independent linear transform
- They combine by simple addition
- Signals & Systems foundation: complex propagation as parallel linear subsystems

# Safety Limiting & The Full Engine Soundscape

## CONDITIONAL TANH SOFT-CLIP Engineering

**safety net** (`audio_engine.py`)

- **Risk:** multiple sources sum  $> \pm 1.0 \implies$  harsh clipping
- **Soft limiter:**  $\mathbf{y}_{\text{mix}} = \tanh(\mathbf{y}_{\text{mix}})$  if  $N_{\text{active}} > 1$
- Single source  $\implies$  strictly linear
- Multiple sources  $\implies$  smooth saturation

## THE FULL ACOUSTIC SANDBOX All

phenomena running live, simultaneously:

- Direct diffracted path bending through doorways
- Muffled transmission bleeding through back wall
- Crisp slapback echoes off adjacent partitions

## The Grand Demo Cue

**DEMO CUT** Activate complex multi-wall scene — hear diffraction, bleed, and echoes live at 44.1 kHz.

## Challenge 8: Material Acoustic Diversity

---

*“Should a heavy velvet curtain and a solid concrete bunker behave the same way? Obviously not — one absorbs treble and lets sound pass through; the other bounces sound like an acoustic mirror.”*

### THE CHALLENGE

How do we support diverse wall materials without introducing dozens of arbitrary, disconnected tuning knobs?

- Moving beyond binary “wall vs. empty grid” geometry
- A unified parameter where reflectivity and transmission loss are inherently coupled

# Multi-Material Wall System: Hard, Medium, Soft

**PALETTE SWATCHES** Walls painted via sidebar swatches (`shared_state.py`):

- **Hard** (concrete):  $\alpha_r=0.90$  – crisp echo, 0.10 transmission
- **Medium** (wood, default):  $\alpha_r=0.60$  – balanced (0.40 transmission)
- **Soft** (curtain):  $\alpha_r=0.25$  – porous (0.75 transmission)

**UNIFIED ACOUSTIC COUPLING**  $\alpha_r$

simultaneously controls three behaviors:

1. Echo gain:  $g_{\text{echo}} \propto \alpha_r$
2. Echo cutoff:  
 $f_c = \max(300, 3500 - (1 - \alpha_r) \cdot 3000)$  Hz
3. Through-wall bleed:  $\tau = (1 - \alpha_r)$

Energy conservation: a material that transmits cannot also produce a loud reflection.

**CONCRETE 0.90**

**WOOD 0.60**

**CURTAIN 0.25**

## Live Demo Cue

**DEMO CUT**

Paint a wall Concrete — loud slap-back. Repaint Curtain — echo dulls, bleed-through brightens.

# System Architecture: Thread Isolation

## STRICT THREAD SEPARATION The two

**real-time worlds** (`shared_state.py`)

- **Main thread** (Pygame, 60 FPS): drag-and-drop, wall painting, grid rendering
- **PortAudio callback** (44.1 kHz,  $\approx 23$  ms budget): hard real-time – must never block or allocate

**The bridge:** `get_snapshot()` grabs a brief lock, deep-copies positions and walls, releases immediately.

## DELIBERATE TRADE-OFF Why not share

the audio thread's geometry cache?

- Cache in `filter_state` is mutated lock-free inside the callback
- Sharing risks priority inversion / buffer underruns
- **Choice:** render thread re-evaluates the cheap geometry functions independently ( $< 1$  ms)

## Architecture Takeaway

*Tiny duplicated CPU work in exchange for zero lock contention and complete thread safety.*

# Live Grid Visualization: Seeing the Acoustics

---

**REAL-TIME OVERLAYS** As users draw walls, the grid shows which phenomenon is active (`render_engine.py`):

- **SOLID TEAL** – primary routed path (Family A)
- **DASHED ORANGE** – through-wall transmission (Family B)
- **MAGENTA RAYS** – echo reflections (Family C) with bounce markers

**AUDIO-VISUAL CORRESPONDENCE** Why this matters for the demo:

- Without visual cues, spatial audio is invisible
- Place a wall → teal line bends, orange dashed line appears
- Box in a source → teal disappears, dashed remains, magenta rays bounce inside

## Presentation Tip

*During the demo, point to the colored path lines to explain which formula is feeding the speakers.*

# Visualizing the Spectrum: Real-Time FFT Panel

## LOG-SPACED FFT BINS Sidebar spectrum

**analyzer** (`viz_engine.py`)

- $|X[k]| = |\text{rfft}(x[n])|$  drawn as animated bars
- **Log binning** (40 Hz  $\rightarrow$  22 kHz): mimics pitch hearing
- **Focus:** 300–4000 Hz (wall filter range)

**MAGNITUDE**  $|X[k]|$

**LOG BINS**

## TIME vs. FREQUENCY DOMAIN

- **Time domain:** waveforms show distance and delay
- **Frequency domain:** spectrum exposes filtering directly
- **Visual proof:** audience sees HF bars drop as wall is added

## Live Demo Cue

**DEMO CUT** Toggle a wall: watch HF bars drop instantly as sound muffles.

# Summary: The Signal Processing Chain

---

## FROM SIGNALS TO SOUNDSCAPES

1. **Distance & panning:** signal scaling, spatial balance
  2. **Per-block FFT lowpass:** discrete Fourier filtering (no IIR)
  3. **Diffraction routing:** 8-dir Yen's  $A^*$  with corner penalty
  4. **Discrete delay lines:** circular buffers for propagation
  5. **Through-wall bleed:** continuous fading, no floor snap
  6. **Multi-wall echoes:** 12-ray fan, top-4 reflectors
  7. **Multi-material:** Hard / Medium / Soft coupling
  8. **Superposition:** direct summation into stereo out
- 

A complete from-scratch layered audio model, built strictly from DFT/FFT, delay operators, and 2D grid search.

# Summary: System Highlights

---

## CAPABILITIES

- **Flexible input:** file sandbox, local mic, multi-device UDP relay (firewall-tested)
- **Dual real-time visualization:** grid path overlays + log-spaced FFT spectrum analyzer
- **Clean architecture:** strict thread isolation via SharedState; zero dead code

## TESTING & QUALITY

- `test_dsp_engine.py` – 24 tests, 117 assertions
- `test_pathfinding.py` – 16 test blocks
- `test_viz_engine.py` & `test_udp_sender.py`
- Modular black-box design; no leftover dead code

3 INPUT MODES

3 WALL FAMILIES

117+ ASSERTIONS

ZERO DEAD CODE

# Live Demonstration

---

## THE SANDBOX IN ACTION

### Interactive Demonstration

We invite you to test, draw, and listen to the engine live.

1. **Move the source:** smooth inverse-distance decay and stereo pan
2. **Drop a wall:** watch high frequencies collapse on the spectrum sidebar
3. **Box in the listener:** smooth fade into through-wall bleed – zero snapping
4. **Switch materials:** bright slapback off Concrete vs. deadened Curtain
5. **Dual reflectors:** two distinct echoes from separate walls at different times

3 INPUT MODES

3 ARRIVAL FAMILIES

117+ PASSING TESTS

LIVE DEMO

# Thank You



Questions & Live Sandbox Exploration